```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.preprocessing import LabelEncoder, StandardScaler,
MinMaxScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import mutual_info_classif, chi2
from sklearn.metrics import mean_squared_error, r2_score,
accuracy_score
```

FASE 1: BUSINESS UNDERSTANDING

Tujuan Studi Kasus: Membangun model untuk memprediksi keselamatan penumpang (0/1).

Target Variable: 'Survived'

FASE 2: DATA UNDERSTANDING (Load & EDA)

```
# 1. Load Data
url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/
titanic.csv"
df = pd.read_csv(url)
print("Dataset loaded. Shape:", df.shape)
```

```
Dataset loaded. Shape: (891, 12)
```

```
df.head(100)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 891,\n  \"fields\": [\
n    {\n      \"column\": \"PassengerId\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 257,\n        \"min\": 1,\n
\"max\": 891,\n        \"num_unique_values\": 891,\n
\"samples\": [\n          710,\n          440,\n          841\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Survived\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
```

```
0,\n        \"min\": 0,\n        \"max\": 1,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          1,\n
0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Pclass\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0,\n        \"min\": 1,\n        \"max\": 3,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          3,\n
1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 891,\n        \"samples\": [\n
\"Moubarek, Master. Halim Gonios (\\\"William George\\\")\",\n
\"Kvillner, Mr. Johan Henrik Johannesson\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Sex\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 2,\n
\"samples\": [\n          \"female\",\n          \"male\"\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Age\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 14.526497332334044,\
n        \"min\": 0.42,\n        \"max\": 80.0,\n
\"num_unique_values\": 88,\n        \"samples\": [\n          0.75,\n
22.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"SibSp\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1,\n        \"min\": 0,\n        \"max\": 8,\n
\"num_unique_values\": 7,\n        \"samples\": [\n          1,\n
0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Parch\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0,\n        \"min\": 0,\n        \"max\": 6,\n
\"num_unique_values\": 7,\n        \"samples\": [\n          0,\n
1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Ticket\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 681,\n        \"samples\": [\n
\"11774\",\n          \"248740\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Fare\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 49.693428597180905,\n
\"min\": 0.0,\n        \"max\": 512.3292,\n
\"num_unique_values\": 248,\n        \"samples\": [\n
11.2417,\n          51.8625\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Cabin\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 147,\n
\"samples\": [\n          \"D45\",\n          \"B49\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Embarked\",\n      \"properties\":
```

{\n   \"dtype\": \"category\",\n   \"num_unique_values\": 3,\n   \"samples\": [\n   \"S\",\n   \"C\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   ]\n}","type":"dataframe","variable_name":"df"}

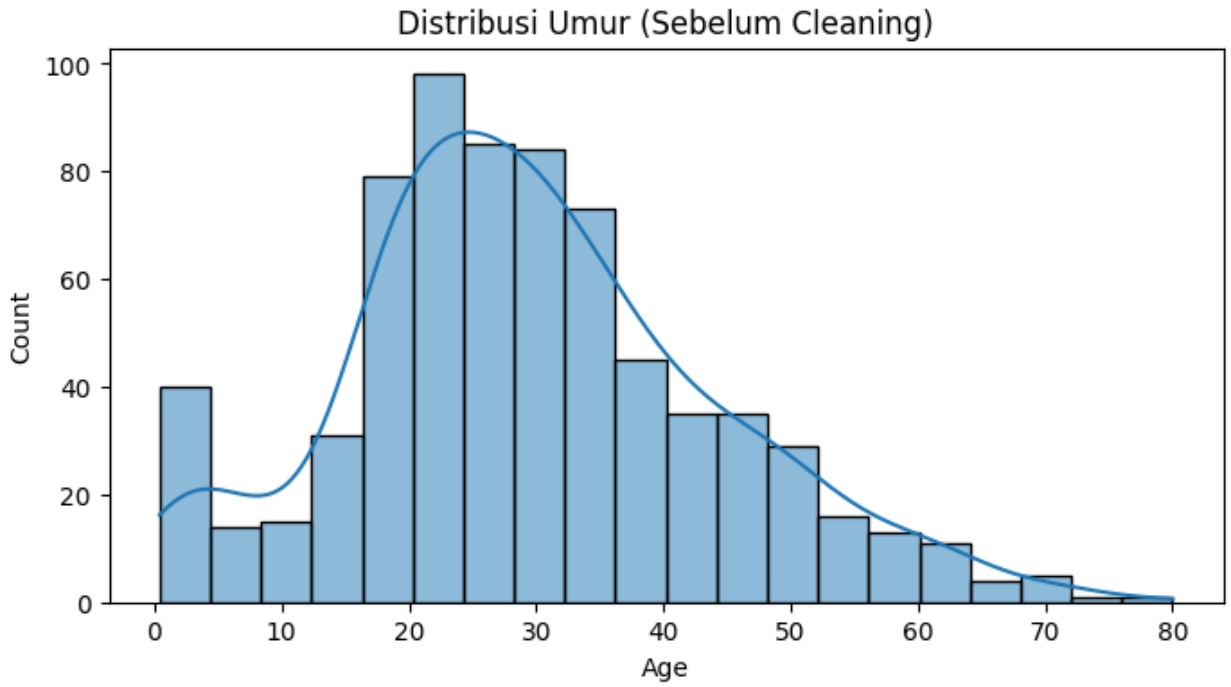
```python
# 2. Describe Data
print("\n--- Statistik Awal ---")
print(df[['Age', 'Fare', 'Survived']].describe())
```

```
--- Statistik Awal ---
              Age        Fare    Survived
count  714.000000  891.000000  891.000000
mean    29.699118   32.204208    0.383838
std     14.526497   49.693429    0.486592
min      0.420000    0.000000    0.000000
25%     20.125000    7.910400    0.000000
50%     28.000000   14.454200    0.000000
75%     38.000000   31.000000    1.000000
max     80.000000  512.329200    1.000000
```

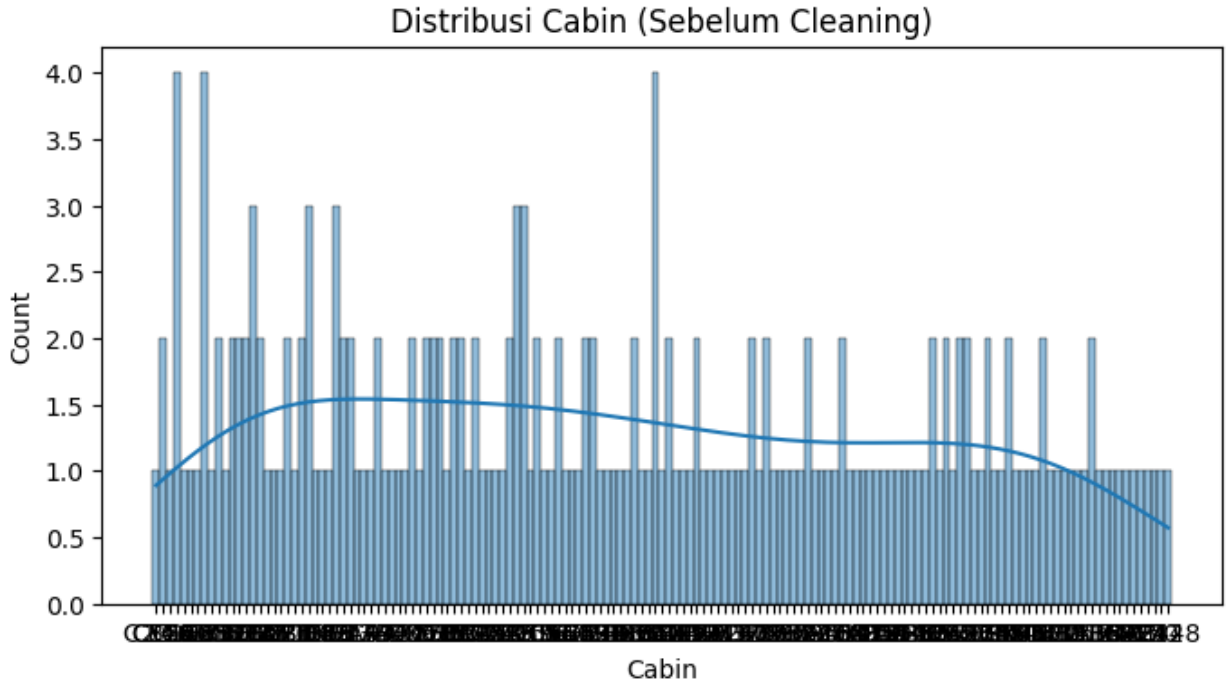
```python
# 3. Cek Missing Value Awal
print("\n--- Cek Missing Value Awal ---")
print(df.isnull().sum())
```

```
--- Cek Missing Value Awal ---
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```python
# 4. Visualisasi Awal (Histogram)
plt.figure(figsize=(8, 4))
sns.histplot(df['Age'].dropna(), bins=20, kde=True)
plt.title('Distribusi Umur (Sebelum Cleaning)')
plt.show()
```




```python
# 4. Visualisasi Awal (Histogram)
plt.figure(figsize=(8, 4))
sns.histplot(df['Cabin'].dropna(), bins=20, kde=True)
plt.title('Distribusi Cabin (Sebelum Cleaning)')
plt.show()
```



FASE 3: DATA PREPARATION (Cleaning & Engineering)

```
# --- 3.0 DATA SEBELUM CLEANING ---
# Simpan kondisi awal untuk visualisasi perbandingan
df_before = df.copy()

# --- 3.1 Data Cleansing (Missing Values) ---
print("Melakukan Handling Missing Values...")
# Handling Missing Values
df['Age'] = df['Age'].fillna(df['Age'].median()) # Imputasi Median
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0]) #
Imputasi Modus
df.drop(columns=['Cabin'], inplace=True) # Drop kolom rusak parah
```

```
Melakukan Handling Missing Values...
```

```
print(df['Age'].value_counts())
```

```
Age
28.00    202
24.00     30
22.00     27
18.00     26
30.00     25
        ...
24.50      1
0.67       1
0.42       1
34.50      1
74.00      1
Name: count, Length: 88, dtype: int64
```

```
print(df['Embarked'].value_counts())
```

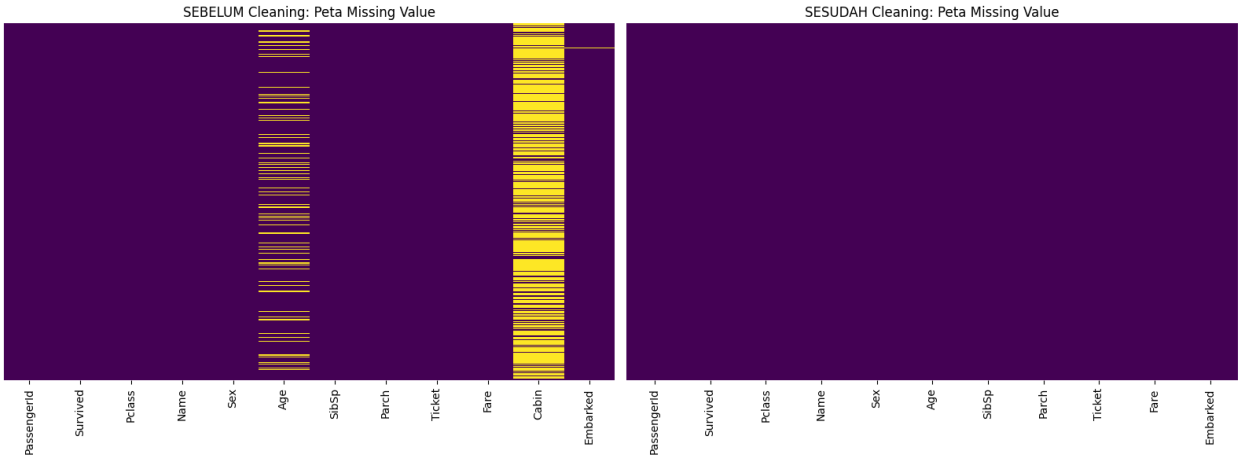
```
Embarked
S    646
C    168
Q     77
Name: count, dtype: int64
```

```
print("Nilai Median Age yang digunakan:", df['Age'].median())
print("Nilai Modus Embarked yang digunakan:", df['Embarked'].mode()
[0])
```

```
Nilai Median Age yang digunakan: 28.0
Nilai Modus Embarked yang digunakan: S
```

```
# ========================================
# Membandingkan df_before vs df (sekarang)
```

```
# ==========================================

# A. VISUALISASI HEATMAP (Pola Missing Value)
fig, axes = plt.subplots(1, 2, figsize=(16, 6))


# Plot Sebelum Cleaning (df_before)
sns.heatmap(df_before.isnull(), cbar=False, cmap='viridis',
yticklabels=False, ax=axes[0])
axes[0].set_title('SEBELUM Cleaning: Peta Missing Value')

# Plot Sesudah Cleaning (df sekarang)
sns.heatmap(df.isnull(), cbar=False, cmap='viridis',
yticklabels=False, ax=axes[1])
axes[1].set_title('SESUDAH Cleaning: Peta Missing Value')

plt.tight_layout()
plt.show()
```



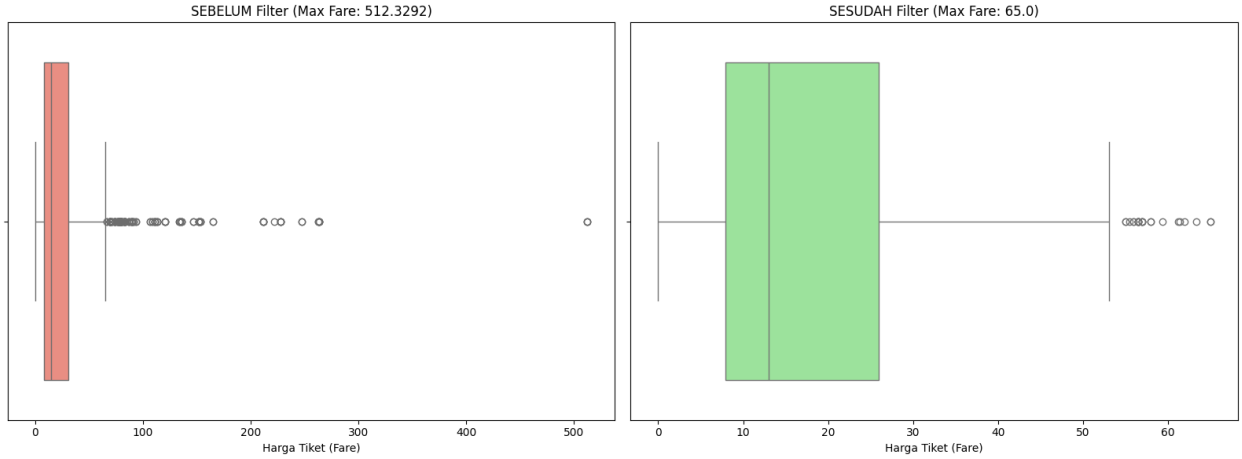

```
df_before_filter = df.copy()

# Hitung IQR (Sesuai kode Anda)
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Batas Bawah: {lower_bound:.2f}")
print(f"Batas Atas : {upper_bound:.2f}")
```

```
Batas Bawah: -26.72
Batas Atas : 65.63
```

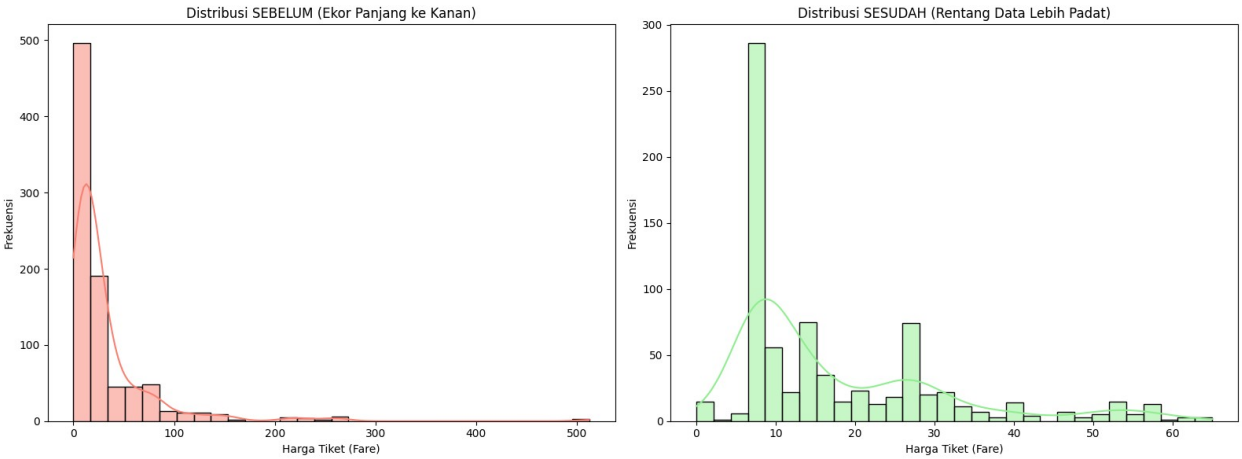
```python
# 3. Cek Missing Value Awal
print("\n--- Cek Missing Value setelah Cleaning ---")
print(df.isnull().sum())
```

```
--- Cek Missing Value setelah Cleaning ---
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

```python
# Filter Outlier
df = df[(df['Fare'] >= lower_bound) & (df['Fare'] <= upper_bound)]
# PENTING: Reset Index agar tidak error saat akses loc
df.reset_index(drop=True, inplace=True)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# 1. Boxplot Sebelum (Kiri)
sns.boxplot(x=df_before_filter['Fare'], ax=axes[0], color='salmon')
axes[0].set_title(f'SEBELUM Filter (Max Fare:
{df_before_filter["Fare"].max()})')
axes[0].set_xlabel('Harga Tiket (Fare)')

# 2. Boxplot Sesudah (Kanan)
sns.boxplot(x=df['Fare'], ax=axes[1], color='lightgreen')
axes[1].set_title(f'SESUDAH Filter (Max Fare: {df["Fare"].max()})')
axes[1].set_xlabel('Harga Tiket (Fare)')

plt.tight_layout()
plt.show()
```




```
# ==========================================
# HISTOGRAM COMPARISON
# ==========================================
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# 1. Histogram Sebelum (Kiri)
sns.histplot(df_before_filter['Fare'], ax=axes[0], color='salmon',
kde=True, bins=30)
axes[0].set_title('Distribusi SEBELUM (Ekor Panjang ke Kanan)')
axes[0].set_xlabel('Harga Tiket (Fare)')
axes[0].set_ylabel('Frekuensi')

# 2. Histogram Sesudah (Kanan)
sns.histplot(df['Fare'], ax=axes[1], color='lightgreen', kde=True,
bins=30)
axes[1].set_title('Distribusi SESUDAH (Rentang Data Lebih Padat)')
axes[1].set_xlabel('Harga Tiket (Fare)')
axes[1].set_ylabel('Frekuensi')

plt.tight_layout()
plt.show()
```



```
# Handling Noise (Simulasi)
# Misal ada umur negatif, kita normalkan
df.loc[df['Age'] < 0, 'Age'] = df['Age'].median()

# --- 3.3 Feature Engineering (Encoding) ---
print("Melakukan Encoding...")
# Label Encoding (Sex)
le = LabelEncoder()
df['Sex_Code'] = le.fit_transform(df['Sex'])
```

```
Melakukan Encoding...
```

```
# One-Hot Encoding (Embarked)
df = pd.get_dummies(df, columns=['Embarked'], prefix='Embark')
for col in df.columns:
    if 'Embark_' in col:
        df[col] = df[col].astype(int)

df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 775,\n  \"fields\": [\n    {\n      \"column\": \"PassengerId\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 260,\n        \"min\": 1,\n\"max\": 891,\n        \"num_unique_values\": 775,\n\"samples\": [\n          494,\n          822,\n          382\n],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"Survived\",\n\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":\n0,\n        \"min\": 0,\n        \"max\": 1,\n\"num_unique_values\": 2,\n        \"samples\": [\n          1,\n0\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\":\n\"Pclass\",\n      \"properties\": {\n        \"dtype\": \"number\",\n\"std\": 0,\n        \"min\": 1,\n        \"max\": 3,\n\"num_unique_values\": 3,\n        \"samples\": [\n          3,\n1\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\":\n\"Name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n\"num_unique_values\": 775,\n        \"samples\": [\n\"Artagaveytia, Mr. Ramon\",\n          \"Lulic, Mr. Nikola\"\n],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"Sex\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n\"samples\": [\n          \"female\",\n          \"male\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"Age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 12.782123201045835,\n        \"min\": 0.42,\n        \"max\": 80.0,\n\"num_unique_values\": 87,\n        \"samples\": [\n          57.0,\n22.0\n        ],\n        \"semantic_type\": \"\",\n
```

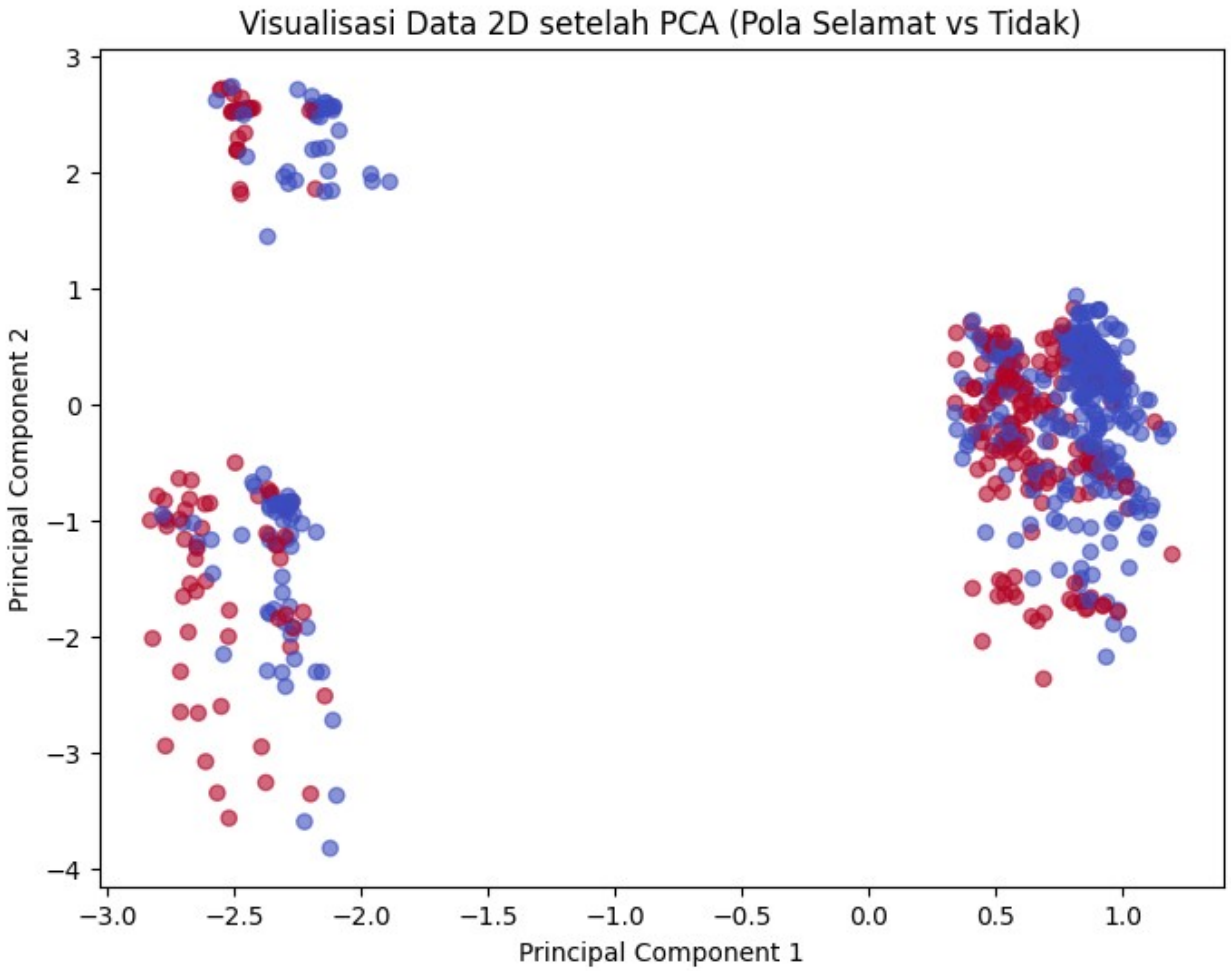
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"SibSp\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0,\n   \"min\": 0,\n   \"max\": 5,\n
\"num_unique_values\": 6,\n   \"samples\": [\n   1,\n
0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Parch\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0,\n   \"min\": 0,\n   \"max\": 6,\n
\"num_unique_values\": 7,\n   \"samples\": [\n   0,\n
1\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Ticket\",\n   \"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 632,\n   \"samples\": [\n
\"110564\",\n   \"250651\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Fare\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 13.57808535378775,\n
\"min\": 0.0,\n   \"max\": 65.0,\n   \"num_unique_values\":
203,\n   \"samples\": [\n   13.0,\n   16.7\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"Sex_Code\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
0,\n   \"min\": 0,\n   \"max\": 1,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   0,\n
1\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Embark_C\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0,\n   \"min\": 0,\n
\"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\":
[\n   1,\n   0\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \"Embark_Q\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0,\n   \"min\": 0,\n
\"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\":
[\n   1,\n   0\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \"Embark_S\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0,\n   \"min\": 0,\n
\"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\":
[\n   0,\n   1\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"df"}

```
# --- 3.4 Data Integration ---
print("Melakukan Data Integration...")
loyalty_data = pd.DataFrame({
    'PassengerId': df['PassengerId'],
    'LoyaltyPoints': np.random.randint(0, 100, size=len(df))
})
df_final = pd.merge(df, loyalty_data, on='PassengerId', how='left')
```

```
Melakukan Data Integration...

# --- 3.5 Scaling (Standarisasi) ---
print("Melakukan Scaling...")
features = ['Age', 'Fare', 'Sex_Code', 'LoyaltyPoints', 'Embark_C',
'Embark_Q', 'Embark_S']
# features = ['Age', 'Fare', 'Sex_Code', 'Embark_C']
target = 'Survived'

X = df_final[features]
y = df_final[target]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(f"Data Preparation Selesai. Shape Data Siap Latih:
{X_scaled.shape}")

Melakukan Scaling...
Data Preparation Selesai. Shape Data Siap Latih: (775, 7)
```

FASE 4: MODELING (Feature Selection & PCA)

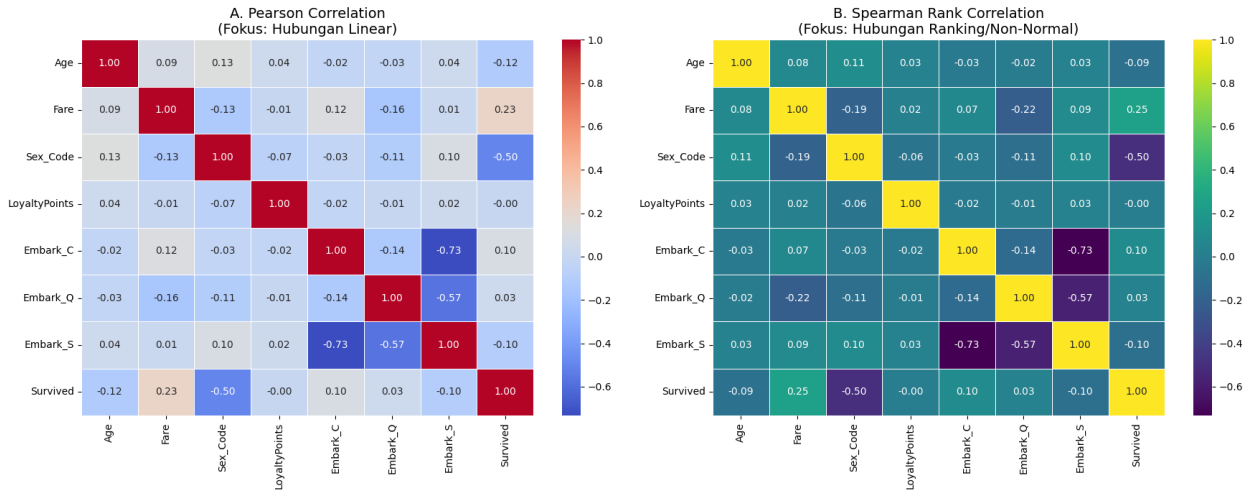
```
# 1. Feature Selection
selector = SelectKBest(score_func=f_classif, k=3)
X_new = selector.fit_transform(X_scaled, y)
selected_indices = selector.get_support(indices=True)
print(f"3 Fitur Terbaik: {[features[i] for i in selected_indices]}")

3 Fitur Terbaik: ['Age', 'Fare', 'Sex_Code']

# 2. PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='coolwarm', alpha=0.6)
plt.title('Visualisasi Data 2D setelah PCA (Pola Selamat vs Tidak)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```



FASE 5: EVALUATION (Inferensial)

```
# Kita tentukan fitur yang akan dianalisis
# Catatan: Kita gunakan data numeric/encoded yang sudah ada di
df_final
cols_to_analyze = features + ['Survived']
curr_df = df_final[cols_to_analyze]

#
==========================================================================
========
# 1. VISUALISASI MATRIKS KORELASI (PEARSON VS SPEARMAN)
#
==========================================================================
========
# Tujuannya: Melihat hubungan antar variabel independen dan hubungan
linear dasar dengan target.

fig, axes = plt.subplots(1, 2, figsize=(18, 7))
```

```
# --- A. Pearson Correlation (Linear) ---
corr_pearson = curr_df.corr(method='pearson')
sns.heatmap(corr_pearson, annot=True, cmap='coolwarm', fmt=".2f",
linewidths=0.5, ax=axes[0])
axes[0].set_title('A. Pearson Correlation\n(Fokus: Hubungan Linear)',
fontsize=14)

# --- B. Spearman Correlation (Rank/Monotonik) ---
corr_spearman = curr_df.corr(method='spearman')
sns.heatmap(corr_spearman, annot=True, cmap='viridis', fmt=".2f",
linewidths=0.5, ax=axes[1])
axes[1].set_title('B. Spearman Rank Correlation\n(Fokus: Hubungan
Ranking/Non-Normal)', fontsize=14)

plt.tight_layout()
plt.show()
```



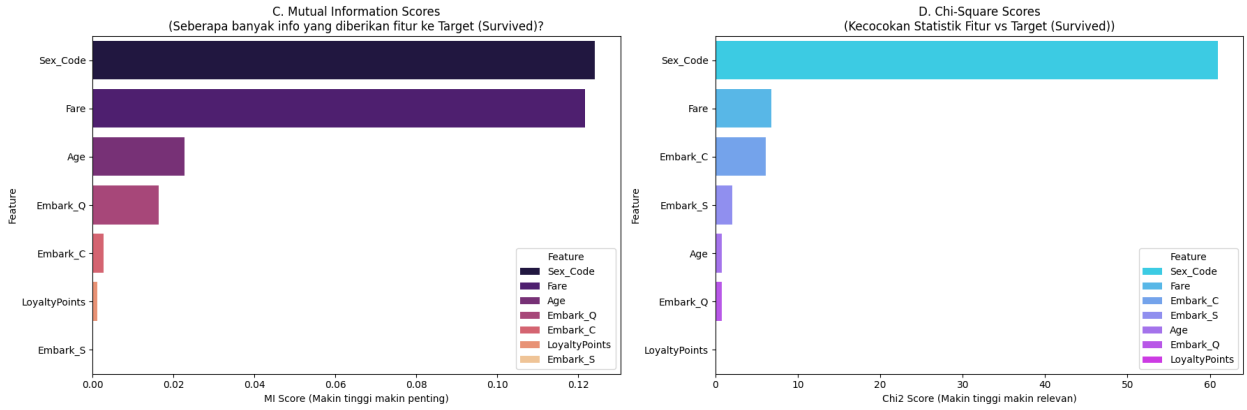

```
#
==========================================================================
========
# 2. FEATURE IMPORTANCE (MUTUAL INFORMATION & CHI-SQUARE)
#
==========================================================================
========
# Tujuannya: Mengetahui seberapa kuat fitur memprediksi 'Survived'
(Target).
# Ini lebih akurat daripada korelasi matriks biasa untuk klasifikasi.

# Siapkan data X (Fitur) dan y (Target)
X_eval = df_final[features]
y_eval = df_final['Survived']

# --- Hitung Mutual Information ---
```

```
# MI menghitung ketergantungan (dependency), termasuk pola non-linear
mi_scores = mutual_info_classif(X_eval, y_eval, random_state=42)
mi_data = pd.DataFrame({'Feature': features, 'MI_Score': mi_scores})
mi_data = mi_data.sort_values(by='MI_Score', ascending=False)

# --- Hitung Chi-Square (Khusus Fitur Non-Negatif) ---
# Kita ambil fitur yang positif saja karena Chi2 tidak menerima nilai
negatif
# (StandardScaler menghasilkan negatif, jadi kita pakai MinMaxScaler
sementara untuk tes ini)
X_pos = MinMaxScaler().fit_transform(X_eval)
chi2_scores, p_values = chi2(X_pos, y_eval)
chi2_data = pd.DataFrame({'Feature': features, 'Chi2_Score':
chi2_scores})
chi2_data = chi2_data.sort_values(by='Chi2_Score', ascending=False)

print("Mutual Information")
print(mi_data)
print("\n Chi Data")
print(chi2_data)
```

```
Mutual Information
         Feature  MI_Score
2       Sex_Code  0.124152
1           Fare  0.121592
0            Age  0.022721
5       Embark_Q  0.016364
4       Embark_C  0.002768
3  LoyaltyPoints  0.001138
6       Embark_S  0.000000

 Chi Data
         Feature  Chi2_Score
2       Sex_Code   60.972788
1           Fare    6.769292
4       Embark_C    6.138455
6       Embark_S    2.000300
0            Age    0.781174
5       Embark_Q    0.748822
3  LoyaltyPoints    0.003030
```

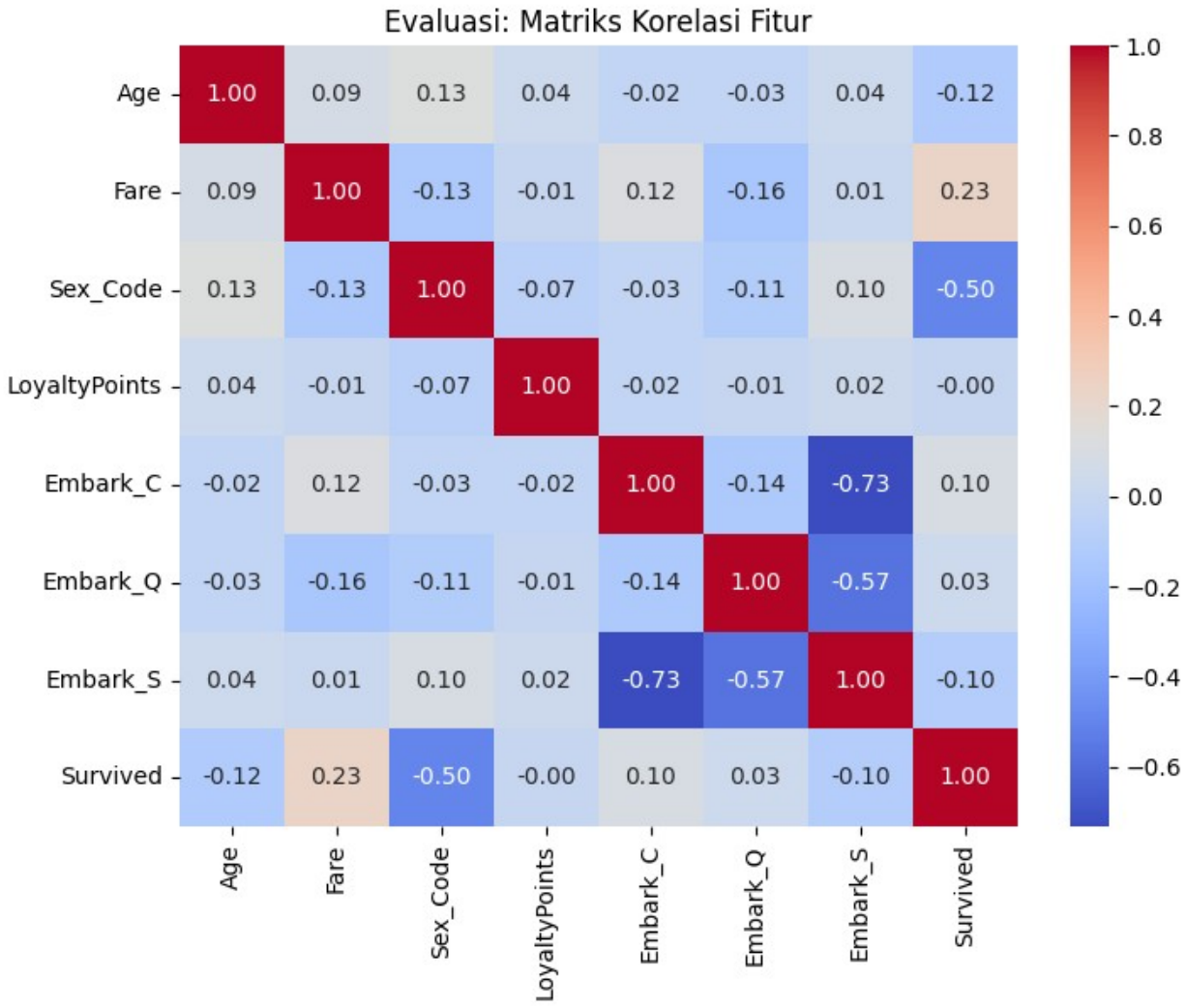
```
#
================================================================================
========
# 3. VISUALISASI FEATURE IMPORTANCE
#
================================================================================
========
fig, axes = plt.subplots(1, 2, figsize=(18, 6))
```

```
# Visualisasi Mutual Information (Bar Plot)
sns.barplot(
    x='MI_Score',
    y='Feature',
    data=mi_data,
    hue='Feature',
    palette='magma',
    legend=True,
    ax=axes[0]
)
axes[0].set_title('C. Mutual Information Scores\n(Seberapa banyak info yang diberikan fitur ke Target (Survived)?')
axes[0].set_xlabel('MI Score (Makin tinggi makin penting)')

# Visualisasi Chi-Square (Bar Plot)
sns.barplot(
    x='Chi2_Score',
    y='Feature',
    data=chi2_data,
    hue='Feature',
    palette='cool',
    legend=True,
    ax=axes[1]
)
axes[1].set_title('D. Chi-Square Scores\n(Kecocokan Statistik Fitur vs Target (Survived))')
axes[1].set_xlabel('Chi2 Score (Makin tinggi makin relevan)')

plt.tight_layout()
plt.show()
```




```
# Kesimpulan Statistik
print("\n--- KESIMPULAN ANALISIS KORELASI ---")
# Pastikan mi_data dan chi2_data
if not mi_data.empty:
    print("1. Fitur paling berpengaruh (Mutual Info):")
```

```
    print(f"   -> {mi_data.iloc[0]['Feature']} (Score:
{mi_data.iloc[0]['MI_Score']:.4f})")
    best_feature = mi_data.iloc[0]['Feature']
    print(f"\nRekomendasi: Fokuskan model Deep Learning pada fitur
'{best_feature}'.")

if not chi2_data.empty:
    print("2. Fitur paling signifikan secara statistik (Chi-Square):")
    print(f"   -> {chi2_data.iloc[0]['Feature']} (Score:
{chi2_data.iloc[0]['Chi2_Score']:.4f})")


--- KESIMPULAN ANALISIS KORELASI ---
1. Fitur paling berpengaruh (Mutual Info):
   -> Sex_Code (Score: 0.1242)

Rekomendasi: Fokuskan model Deep Learning pada fitur 'Sex_Code'.
2. Fitur paling signifikan secara statistik (Chi-Square):
   -> Sex_Code (Score: 60.9728)

# Uji Hipotesis
age_survived = df_final[df_final['Survived'] == 1]['Age']
age_died = df_final[df_final['Survived'] == 0]['Age']

t_stat, p_val = stats.ttest_ind(age_survived, age_died)
print(f"Hasil T-Test Age vs Survived (P-value): {p_val:.4f}")

if p_val < 0.05:
    print("Kesimpulan: Umur berpengaruh SIGNIFIKAN terhadap
keselamatan.")
else:
    print("Kesimpulan: Umur TIDAK berpengaruh signifikan.")

Hasil T-Test Age vs Survived (P-value): 0.0010
Kesimpulan: Umur berpengaruh SIGNIFIKAN terhadap keselamatan.

# Korelasi Matrix
plt.figure(figsize=(8, 6))
correlation = df_final[features + ['Survived']].corr()
sns.heatmap(correlation, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Evaluasi: Matriks Korelasi Fitur')
plt.show()
```



FASE 6: PREDICTIVE MODELING (Logistic Regression & KNN)

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression # Import library
baru
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

print("\n" + "="*50)
print("PREDICTIVE MODELING (Logistic Regression & KNN)")
print("="*50)

# 1. Split Data (80% Train, 20% Test)
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)
print(f"Data dibagi menjadi: Train ({X_train.shape[0]}) dan Test
({X_test.shape[0]})")



================================================
PREDICTIVE MODELING (Logistic Regression & KNN)
================================================
Data dibagi menjadi: Train (620) dan Test (155)

# ========================================
# A. MODEL 1: LOGISTIC REGRESSION
# ========================================
print("\n--- A. Training Logistic Regression ---")
# Logistic Regression otomatis menangani klasifikasi
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)

# Prediksi
y_pred_log = log_reg.predict(X_test)

# Evaluasi
acc_log = accuracy_score(y_test, y_pred_log)
print(f"Akurasi Logistic Regression: {acc_log:.4f}")

print("\nKoefisien (Pengaruh Fitur):")
# Koefisien positif = Meningkatkan peluang selamat
# Koefisien negatif = Menurunkan peluang selamat
for feat, coef in zip(features, log_reg.coef_[0]):
    print(f" - {feat}: {coef:.4f}")

print("\nClassification Report (Logistic):")
print(classification_report(y_test, y_pred_log))


--- A. Training Logistic Regression ---
Akurasi Logistic Regression: 0.7484

Koefisien (Pengaruh Fitur):
 - Age: -0.1430
 - Fare: 0.4240
 - Sex_Code: -1.0653
 - LoyaltyPoints: -0.0872
 - Embark_C: 0.1062
 - Embark_Q: -0.0053
 - Embark_S: -0.0843

Classification Report (Logistic):
              precision    recall  f1-score   support
```

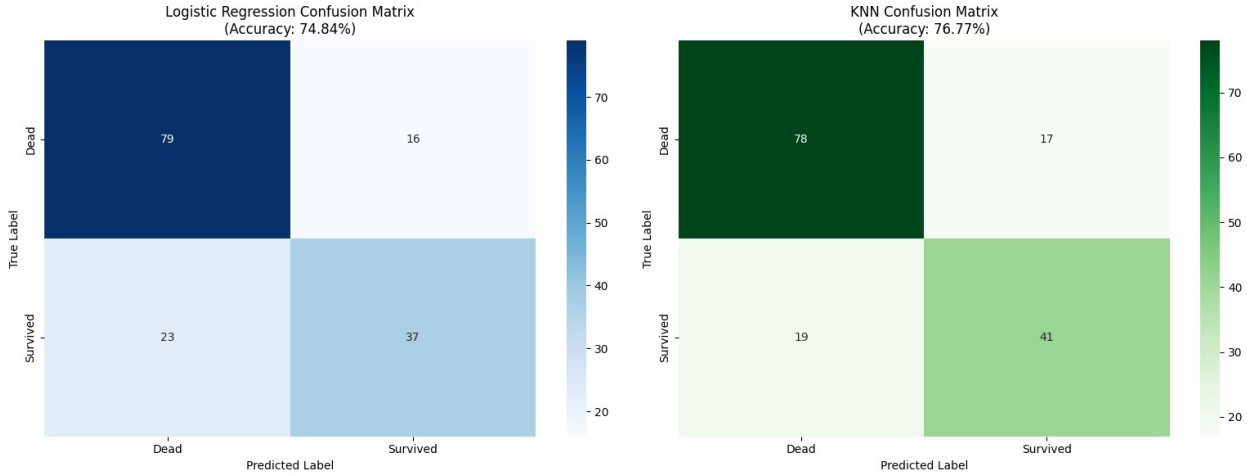
```
           0       0.77      0.83      0.80        95
           1       0.70      0.62      0.65        60

    accuracy                           0.75       155
   macro avg       0.74      0.72      0.73       155
weighted avg       0.74      0.75      0.75       155
```

```
# ========================================
# MODEL 2: K-NEAREST NEIGHBORS (KNN)
# ========================================
print("\n--- B. Training K-Nearest Neighbors (KNN) ---")
# K=5
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Prediksi
y_pred_knn = knn.predict(X_test)

# Evaluasi
acc_knn = accuracy_score(y_test, y_pred_knn)
print(f"Akurasi KNN (k=5): {acc_knn:.4f}")
print("\nClassification Report KNN:")
print(classification_report(y_test, y_pred_knn))
```

```
--- B. Training K-Nearest Neighbors (KNN) ---
Akurasi KNN (k=5): 0.7677

Classification Report KNN:
              precision    recall  f1-score   support

           0       0.80      0.82      0.81        95
           1       0.71      0.68      0.69        60

    accuracy                           0.77       155
   macro avg       0.76      0.75      0.75       155
weighted avg       0.77      0.77      0.77       155
```

```
# ========================================
# C. VISUALISASI PERBANDINGAN (CONFUSION MATRIX)
# ========================================
# Bandingkan Confusion Matrix kedua model
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# 1. Visualisasi Logistic Regression
cm_log = confusion_matrix(y_test, y_pred_log)
sns.heatmap(cm_log, annot=True, fmt='d', cmap='Blues', ax=axes[0])
```

```
axes[0].set_title(f'Logistic Regression Confusion Matrix\n(Accuracy:
{acc_log:.2%})')
axes[0].set_xlabel('Predicted Label')
axes[0].set_ylabel('True Label')
axes[0].set_xticklabels(['Dead', 'Survived'])
axes[0].set_yticklabels(['Dead', 'Survived'])

# 2. Visualisasi KNN
cm_knn = confusion_matrix(y_test, y_pred_knn)
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Greens', ax=axes[1])
axes[1].set_title(f'KNN Confusion Matrix\n(Accuracy: {acc_knn:.2%})')
axes[1].set_xlabel('Predicted Label')
axes[1].set_ylabel('True Label')
axes[1].set_xticklabels(['Dead', 'Survived'])
axes[1].set_yticklabels(['Dead', 'Survived'])

plt.tight_layout()
plt.show()
```




```
# Kesimpulan Akhir
print("\n" + "="*50)
print(" KESIMPULAN MODEL")
print("="*50)
if acc_knn > acc_log:
    print(f"Model KNN lebih unggul dengan akurasi {acc_knn:.2%}")
else:
    print(f"Model Logistic Regression lebih unggul dengan akurasi
{acc_log:.2%}")
```

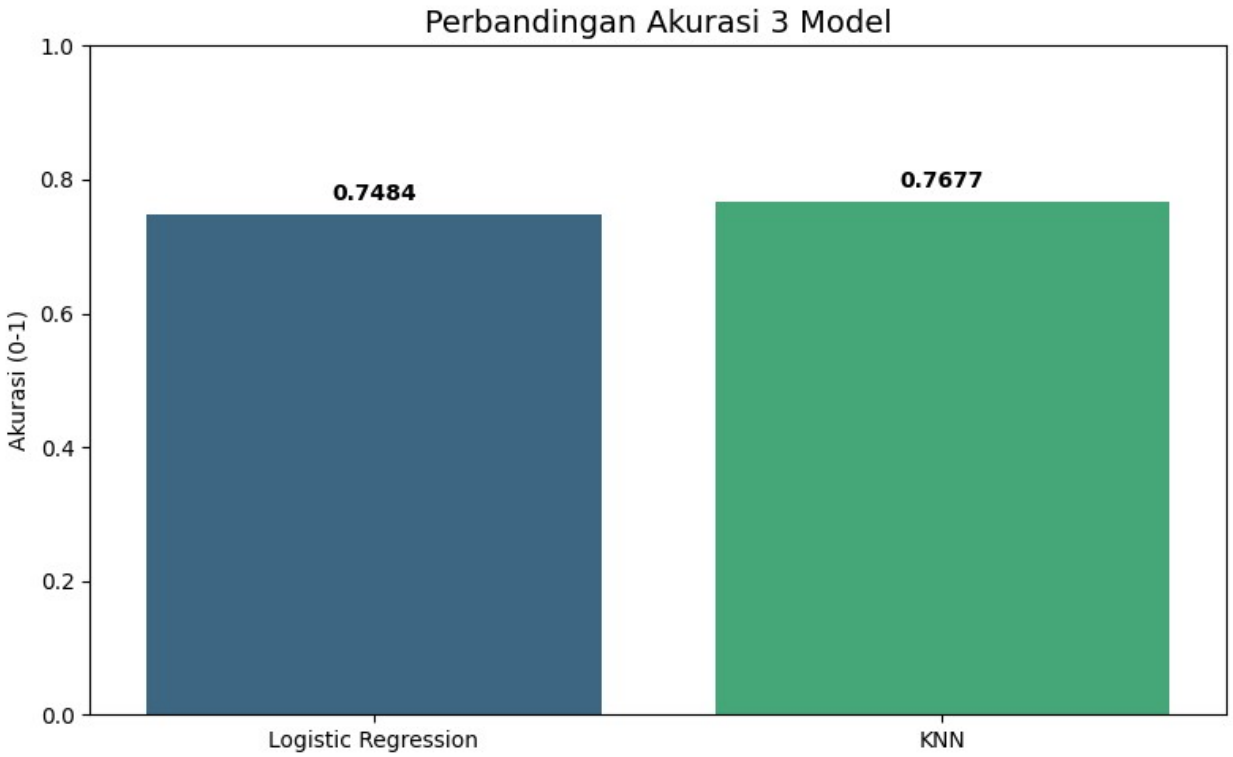
```
==================================================
 KESIMPULAN MODEL
==================================================
Model KNN lebih unggul dengan akurasi 76.77%
```

FASE 6.1: PERBANDINGAN MODEL

```
# 1. Buat DataFrame
comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'KNN'],
    'Accuracy': [acc_log, acc_knn]
})

print("\n--- PERBANDINGAN 2 MODEL ---")
print(comparison.sort_values(by='Accuracy', ascending=False))
```

```
--- PERBANDINGAN 2 MODEL ---
                Model  Accuracy
1                 KNN  0.767742
0  Logistic Regression  0.748387
```

```
# 4. Visualisasi
plt.figure(figsize=(8, 5))

sns.barplot(
    x='Model',
    y='Accuracy',
    data=comparison,
    hue='Model',
    legend=False,
    palette='viridis'
)

plt.ylim(0, 1)
plt.title('Perbandingan Akurasi 3 Model', fontsize=14)
plt.ylabel('Akurasi (0-1)')
plt.xlabel('')

# Menambahkan angka di atas batang
for index, row in comparison.iterrows():
    plt.text(index, row.Accuracy + 0.02, f"{row.Accuracy:.4f}",
ha='center', color='black', fontweight='bold')

plt.tight_layout()
plt.show()
```


Perbandingan Akurasi 3 Model
Akurasi (0-1)
1.0
0.8
0.6
0.4
0.2
0.0
0.7484
0.7677
Logistic Regression
KNN